Table of Contents

Table of Figures

EXECUTIVE SUMMARY

This project develops a healthcare database system that transforms the public ***Diabetes 130-US Hospitals for Years 1999–2008*** dataset from the University of California, Irvine (UCI) Machine Learning Repository from a single wide comma-separated values (CSV) file into a normalized, auditable Structured Query Language (SQL) database. The goal is to design a schema and implementation that support realistic modelling of diabetic inpatient care, with particular emphasis on drug utilization and readmission risk, while satisfying the formal requirements of the CPSC 500 – SQL Databases final project.

The system is built around a 15-table SQL schema that models patients, encounters, readmission outcomes, diagnoses, medications, providers, admission and discharge characteristics, and audit trails. Core clinical entities (patients, encounters, outcomes) are surrounded by lookup tables and three junction tables (encounter_drugs, encounter_diagnoses, encounter_providers) that implement many-to-many relationships. The schema is normalized to Third Normal Form (3NF), avoiding redundancy while remaining intuitive to query. Design decisions are documented and justified in terms of data integrity, extensibility, and analytical usefulness.

Beyond conventional relational structures, the database incorporates semi-structured data in two formats:

- **JavaScript Object Notation (JSON)** for flexible fields such as long-term risk factors and encounter notes.
- **eXtensible Markup Language (XML)** for hierarchical drug monographs, including warnings and contraindications.

These choices allow the schema to store rich clinical context without sacrificing relational clarity. The project demonstrates how JSON and XML fields can be updated and queried using SQL functions, enabling more advanced decision-support scenarios.

The implementation includes complete coverage of Data Definition Language (DDL), Data Manipulation Language (DML), and Data Query Language (DQL): table creation scripts, foreign-key constraints, indexes, data-loading pipelines from a curated, cleaned encounter subset executed through the SQL script, and representative INSERT, UPDATE, and DELETE operations. More advanced database objects views, stored procedures, and triggers are used to

encapsulate common logic and enforce audit requirements. For example, triggers write to an audit_logs table whenever key outcomes or drug exposures change, and views summarise drug-level readmission rates and patient encounter histories.

A central goal of the project is to support machine learning (ML) and analytics. From the normalized schema, three ML-ready datasets are extracted:

- A classification dataset for predicting early readmission.
- A regression dataset for modelling length of stay (LOS).
- A clustering dataset for segmenting patients by risk profile and utilisation patterns.

These datasets are consumed in a Python notebook using a MySQL connector, where basic models and visualisations demonstrate the database's suitability as a backend for predictive analytics.

Overall, the project delivers a design and implementation of a healthcare database system for diabetic inpatient care that is:

- Structurally sound (3NF, clear primary key and foreign key relationships, junction tables for many-to-many links).
- Flexible (JSON and XML support for semi-structured clinical data).
- Auditable (user accounts and audit logs with database triggers).
- Analytics-ready (views, complex queries, and ML datasets integrated with Python).

The report documents the full lifecycle, from requirements and conceptual design to physical implementation, complex querying, and analytical integration - showing how a carefully designed SQL database can support both day-to-day clinical queries and advanced modelling of drug utilization and readmission risk.

1.0 Introduction & System Overview

1.1 Motivation and Project Goals

The starting point for this project is the Diabetes 130-US Hospitals for Years 1999–2008 dataset from the University of California, Irvine (UCI) Machine Learning Repository. In its original form, the dataset is a single flat comma-separated values (CSV) file with more than 50 columns describing hospital encounters for patients with diabetes. While this layout is convenient for quick analysis in spreadsheets or notebooks, it is not well suited for:

- Enforcing data integrity and relationships between entities such as patients, encounters, diagnoses, drugs, and outcomes.
- Supporting reusable views, stored procedures, and triggers for real-world workflows.
- Providing clean, documented datasets for machine learning (ML) over time.

The main goal of the project is to redesign this flat dataset into a structured healthcare database system focused on diabetic inpatient care, with a particular emphasis on drug utilization and readmission risk. The system should look and behave like a small operational data platform: properly normalized, constrained, auditable, and capable of supplying downstream ML and analytics tasks.

Concretely, the project aims to:

- Design a normalized Structured Query Language (SQL) schema with at least ten interrelated tables, including one-to-one (1:1), one-to-many (1:N), and many-to-many (M:N) relationships.
- Implement the schema using Data Definition Language (DDL) with appropriate primary keys, foreign keys, and constraints.
- Populate the schema using DML from a curated, cleaned encounter subset.
- Demonstrate Data Query Language (DQL) through complex queries that answer realistic questions about diabetic inpatient care.
- Integrate JavaScript Object Notation (JSON) and eXtensible Markup Language (XML) fields for semi-structured clinical data.
- Produce and consume ML-ready datasets for classification, regression, and clustering using a Python notebook.

1.2 High-Level System Overview

At a high level, the implemented system can be viewed as a pipeline that turns a raw CSV file into a structured, queryable, and analytics ready database:

- **Source and Staging**
  - The group prepared a cleaned encounter subset externally and loaded the normalized tables directly via scripted SQL INSERT/UPDATE while preserving FK integrity.
  - This staging layer is used for basic validation and transformation but is not queried directly for analysis.
- **Normalized Core Schema**
  - The normalized schema introduces distinct tables for key entities:
    - patients - long-term demographic and risk factor information.
    - encounters - individual hospital visits, including admission and specialty details and temporal age/weight bands.
    - outcomes - readmission category and binary readmission flag linked one-to-one with encounters.
  - Clinical reference data are stored in tables such as drugs, diagnosis_codes, providers, and medical_specialties.
  - Many-to-many relationships are represented via junction tables: encounter_drugs, encounter_diagnoses, and encounter_providers.
  - Administrative metadata is captured in lookup tables (admission_types, admission_sources, discharge_dispositions) and in system tables (user_accounts, audit_logs).
- **Advanced Logic and Auditing**
  - Views provide reusable perspectives such as drug-level outcome summaries and patient encounter histories.
  - Stored procedures encapsulate multi-step operations, for example adding an encounter together with its outcome and initial drug exposures.
  - Triggers write audit records to audit_logs whenever critical clinical facts (such as readmission status or medication exposure) are modified.
- **Analytics and Machine Learning Integration**
  - From the core schema, the project defines SQL queries and views that produce three ML-ready datasets: a classification dataset for early readmission, a

regression dataset for length of stay (LOS), and a clustering dataset for patient risk segmentation.

- These datasets are loaded into a Python notebook using a MySQL connector, where basic models and visualisations are used to confirm that the schema supports downstream analytics.

2.0 Data and Conceptual Design

2.1 Source Dataset and Preparation Approach

This project is based on the public "Diabetes 130-US Hospitals for Years 1999–2008" dataset from the University of California, Irvine (UCI) Machine Learning Repository. The original data are provided as a single wide comma-separated values (CSV) file where each row represents one hospital encounter for a patient with diabetes.

The raw file contains more than 50 columns, including:

- **Identifiers and demographics**
  - encounter_id: unique identifier for each hospital visit.
  - patient_nbr: identifier that links multiple encounters for the same patient.
  - race, gender.
  - age: stored as bands (for example, "[50-60)", "[70-80)" rather than exact years).
  - weight: also stored as broad categories when available.
- **Administrative and utilisation variables**
  - admission_type_id, discharge_disposition_id, admission_source_id.
  - time_in_hospital (days).
  - number_outpatient, number_emergency, number_inpatient (prior utilisation counts).
  - payer_code, medical_specialty.
- **Clinical measurements and diagnostic codes**
  - max_glu_serum, A1Cresult.
  - diag_1, diag_2, diag_3: International Classification of Diseases (ICD) codes for primary and secondary diagnoses.
- **Medication exposure flags**
  - A set of columns for specific drugs (for example metformin, insulin, glipizide, and many others).
  - Each drug column stores categorical values such as "No", "Steady", "Up", or "Down" to indicate exposure and dosage changes.
- **Outcome**
  - readmitted: a categorical variable indicating "NO", "<30", or ">30" days until readmission.

From a database design perspective, this structure is convenient for quick analysis but problematic for a maintainable healthcare information system:

- Multiple concepts; patients, encounters, diagnoses, drugs, and outcomes are mixed into one table.
- Repeating groups exist (diagnosis columns diag_1–diag_3, many medication columns) instead of normalised relationships.
- Age and weight are stored as bands without explicit handling of time-varying demographics.
- There is no representation of users, audit trails, or semi-structured clinical context.

To address these issues, the project uses a curated, cleaned encounter dataset derived from the original CSV. Instead of introducing an in-database raw staging table, the final implementation prioritises a fully reproducible build by loading the normalised schema directly through the submitted SQL script. The cleaned subset supports the same logical transformation goals (patients, encounters, lookups, diagnoses, drugs, outcomes) without requiring a raw_encounters layer in the final deliverable.

In the normalised design, patient_id serves as the primary key for the patients table. In this implementation, patient_id is populated using stable identifiers from the curated subset to maintain a clean and repeatable final script. This approach preserves relational integrity and supports efficient joins and foreign key relationships within the submission scope.

For practical reasons, the project uses a curated subset of 250 encounters for loading and testing. This exceeds the minimum requirement for realistic record population and provides sufficient variety of diagnoses, medications, admission contexts, and readmission outcomes to demonstrate the full relational model, advanced SQL objects, and semi-structured JSON/XML features. Where the source dataset does not supply required entities (e.g., individual provider identities), small controlled synthetic additions are introduced to operationalise the ERD's many-to-many clinical relationships.

2.2 From Requirements to Conceptual Model

The conceptual design of the database is driven by both the formal course requirements and the practical needs implied by the UCI diabetic dataset. The key functional requirements can be summarised as:

- **Represent patients and multiple encounters**

  - A single patient may appear in the raw data many times via patient_nbr.
  - The system must support multiple hospital encounters per patient and preserve the link between them, while mapping the original patient_nbr to an internal patient_id surrogate key.

- **Capture rich encounter level context**
  - Each encounter needs to record its admission type, source, discharge disposition, medical specialty, temporal age and weight bands, prior utilisation counts, and length of stay.
- **Model diagnoses, medications, and providers in a flexible way**
  - The flat columns diag_1, diag_2, diag_3 must be generalised into a structure that can hold any number of diagnoses per encounter.
  - The many drug flag columns must be represented as a proper relationship between encounters and drug entities, rather than as dozens of separate fields.
  - Healthcare providers involved in each encounter should be tracked.
- **Store outcomes explicitly**
  - Readmission information (category and binary flag) should be attached clearly to each encounter, in a form that is easy to query and audit.
- **Support reference and lookup data**
  - Admission types, admission sources, discharge dispositions, diagnosis codes, and medical specialties should be normalised into separate tables to avoid duplication and ensure consistent coding.
- **Integrate semi-structured clinical information**
  - The design must include at least one **JSON** field and one **XML** field to store richer structures such as patient risk factors, encounter notes, and drug monographs.
- **Provide basic security and auditing**
  - The system should include user accounts and audit logs so that changes to critical clinical facts (for example, readmission status or medication exposure) can be tracked, including automated changes executed by system processes.
- **Enable downstream analytics and machine learning (ML)**
  - The schema should make it straightforward to construct datasets for classification, regression, and clustering tasks related to diabetic inpatient care.

Translating these requirements into a conceptual model leads to the following main entity types:

- **Patient** - represents an individual person with diabetes. Includes demographic attributes (race, gender), a mapping from the original patient_nbr to an internal patient_id, and a JSON risk profile.
- **Encounter** - represents a single hospital stay, linked to exactly one patient via patient_id. Captures admission and discharge metadata, temporal age and weight bands, prior utilisation counts, and time in hospital.
- **Outcome** - represents the readmission result for a specific encounter, including a categorical readmission status ("NO", "<30", ">30") and a derived binary readmission flag.
- **Drug** - represents a medication that may be used during an encounter. Includes code, generic name, drug class, and an XML monograph describing warnings and contraindications.
- **Diagnosis** - represents an International Classification of Diseases (ICD) diagnosis code, with a description.
- **Provider** - represents a healthcare professional involved in care (for example, attending physician), including name, specialty, provider type, and licence number.
- **Medical Specialty** - represents high-level clinical departments such as cardiology, surgery, or endocrinology.
- **Admission Type, Admission Source, Discharge Disposition** - represent normalised lookup values for how the patient came into hospital and where they went after discharge.
- **User Account** - represents a logical system user, including both human users (for example, administrator or analyst accounts) and a special **"System"** user used to attribute automated operations (for example, trigger-based updates).
- **Audit Log** - records changes made to key tables, linking each change back to a user account and, where relevant, an encounter.

Certain relationships are inherently **many-to-many** in the clinical context:

- An encounter can involve multiple diagnoses, and the same diagnosis can appear in many encounters.
- An encounter can involve multiple drugs, and the same drug can be used across many encounters.

- An encounter can involve multiple providers, and a provider can participate in many encounters.

These are handled conceptually by three junction entities:

- **Encounter-Drug** - links an encounter to a drug with an exposure status.
- **Encounter-Diagnosis** - links an encounter to a diagnosis with an order (primary, secondary, etc.).
- **Encounter-Provider** - links an encounter to a provider with a role (attending, consulting, assisting).

This conceptual model separates concerns clearly: patients are distinct from encounters; encounters are distinct from outcomes; diagnoses, drugs, and providers are reusable reference entities; and many-to-many relationships are implemented explicitly rather than through repeating columns or multiple flags.

2.3 High-Level Conceptual Relationships

Before translating the conceptual model into a full entity-relationship diagram (ERD) and physical schema, it is useful to describe the most important relationships in words.

- **Patient-Encounter (one-to-many)**
  - A **Patient** can have **many Encounters** over time.
  - Each **Encounter** is associated with **exactly one Patient**, using an internal patient_id surrogate key that is derived from the original patient_nbr in the source data.
- **Encounter-Outcome (one-to-one)**
  - Each encounter has at most one **Outcome** describing its readmission status.
  - This is modelled as a **one-to-one (1:1)** relationship where the primary key of outcomes is also a foreign key to encounters.
  - This separation allows encounters to exist even if the outcome is not yet known or has not yet been recorded.
- **Encounter-Diagnosis (many-to-many via junction)**
  - An **Encounter** can have multiple **Diagnoses**, and the same **Diagnosis** may appear in many Encounters.

  - Conceptually, this is a **many-to-many (M:N)** relationship realised through the **Encounter-Diagnosis** junction entity, which also stores the diagnosis order (primary, secondary, etc.).
- **Encounter-Drug (many-to-many via junction)**
  - An **Encounter** can be associated with several **Drugs**, and the same **Drug** may be prescribed in many Encounters.
  - This is represented through the **Encounter–Drug** junction entity, which also stores the exposure status (for example, "No", "Steady", "Up", "Down").

- **Encounter-Provider (many-to-many via junction)**
  - An **Encounter** can involve multiple **Providers**, and a **Provider** may work on many Encounters.
  - This is captured through the **Encounter–Provider** junction entity, which includes the provider's role on that encounter.
- **Encounter-Lookup Tables (one-to-many)**
  - Each Encounter refers to exactly one **Admission Type**, one **Admission Source**, one **Discharge Disposition**, and one **Medical Specialty**, while each of these lookup values can be used by many encounters.
  - These are classic **one-to-many (1:N)** relationships and remove repeated codes from the raw flat data.
- **User Account-Audit Log (one-to-many)**
  - A **User Account** can be responsible for many **Audit Log** entries.
  - An **Audit Log** row optionally references a User (for system-generated actions this may be the special "System" account or null, depending on implementation) and optionally an Encounter or other table.
  - This structure allows the system to record who changed what and when without constraining auditing to a single table.

Altogether, these relationships form the conceptual backbone of the database. In the next chapter, this conceptual model is translated into a full logical design and physical schema, including attributes, keys, junction tables, and the representation of JSON and XML fields in the final entity–relationship diagram.

3.0 Database Design and Entity–Relationship Diagram

3.1 Logical Schema Overview

The conceptual model from Chapter 2 is implemented as a 15-table Structured Query Language (SQL) schema. The tables are organised into five logical groups:

- **Core clinical entities**
  - patients
  - encounters
  - outcomes
- **Clinical reference data**
  - drugs
  - diagnosis_codes
  - providers
  - medical_specialties
- **Junction (many-to-many) tables**
  - encounter_drugs
  - encounter_diagnoses
  - encounter_providers
- **Lookup tables for administrative metadata**
  - admission_types
  - admission_sources
  - discharge_dispositions
- **System and audit tables**
  - user_accounts
  - audit_logs

A curated cleaned encounter subset is used as the source for scripted population

3.1.1 Logical schema overview (concise description of each table)

- patients – Master record for each person with diabetes, including demographics, original patient_id, age and weight bands, and a JavaScript Object Notation (JSON) risk profile.

- encounters – One row per hospital encounter; central "fact-like" table containing admission and discharge metadata, temporal age and weight bands, utilisation counts, and a JSON encounter note field.
- outcomes – One-to-one (1:1) extension of encounters holding readmission category and binary readmission flag.
- drugs – Reference list of medications with codes, generic names, drug classes, and an eXtensible Markup Language (XML) drug monograph.
- diagnosis_codes – Reference table for International Classification of Diseases (ICD) codes and descriptions.
- providers – Reference table for healthcare professionals (names, specialties, provider types, licence numbers).
- medical_specialties – Lookup table for high-level clinical specialties/departments.
- admission_types, admission_sources, discharge_dispositions – Lookup tables for admission and discharge metadata.
- encounter_drugs – Junction table linking encounters to drugs with exposure status.
- encounter_diagnoses – Junction table linking encounters to diagnosis codes with diagnosis order.
- encounter_providers – Junction table linking encounters to providers with their role in the encounter.
- user_accounts – System user registry (including an internal "System" user) with roles and credentials.
- audit_logs – Record of data modification events, linking actions back to user accounts and, where applicable, to encounters.

The overall design principle is that encounters acts as a central hub, with dimension-like tables (patients, diagnoses, drugs, providers, specialties, admission metadata) connected around it. Junction tables implement many-to-many (M:N) clinical relationships, and system tables provide cross-cutting concerns such as user management and auditing.

3.2 Entity–Relationship Diagram and Notation

The logical schema is captured visually in an **entity–relationship diagram (ERD)** generated using dbdiagram.io. The ERD groups tables into their functional areas and highlights primary key (PK) and foreign key (FK) relationships.

To make the diagram easier to interpret, the following notation is used:

- **Primary keys** are underlined or marked with key icons.
- **Foreign keys** are marked with a link icon and draw arrows to their parent tables.
- **One-to-many (1:N)** relationships are drawn from the "one" side (PK) to the "many" side (FK).
- **One-to-one (1:1)** is implemented by using the same PK value in both tables (encounters and outcomes).
- **Many-to-many (M:N)** relationships are implemented via junction tables with composite primary keys (for example, encounter_id + drug_id in encounter_drugs).

At a high level, the ERD can be read as follows:

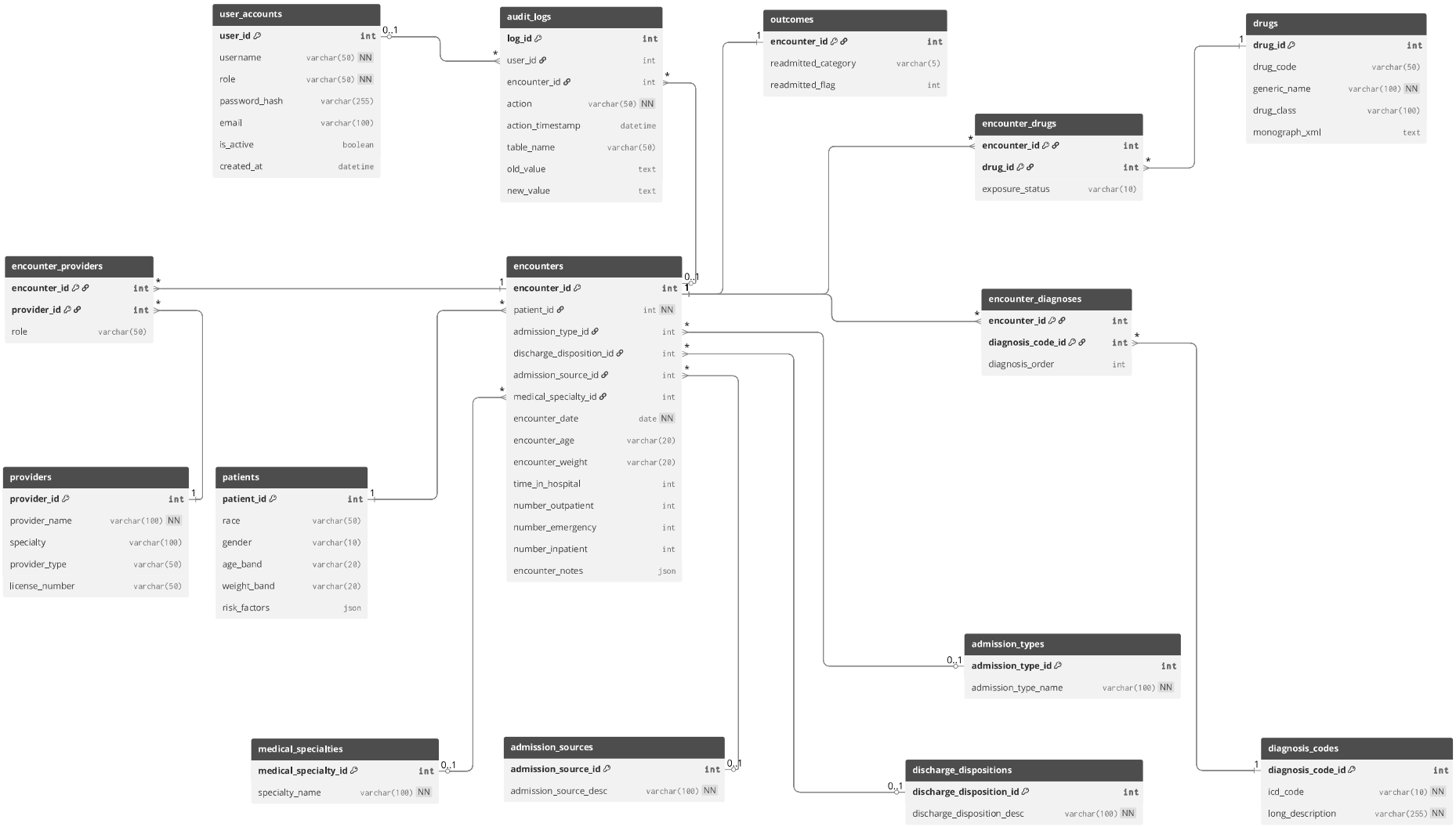
- patients (1) ──< (N) encounters
- encounters (1) ── (1) outcomes
- encounters (M) ─< encounter_drugs >─ (N) drugs
- encounters (M) ─< encounter_diagnoses >─ (N) diagnosis_codes
- encounters (M) ─< encounter_providers >─ (N) providers
- admission_types, admission_sources, discharge_dispositions, and medical_specialties each have a one-to-many relationship with encounters.
- user_accounts (1) ──< (N) audit_logs, with optional foreign keys to encounters and other target tables as needed.

The ERD also highlights semi-structured fields:

- patients.risk_factors (JSON)
- encounters.encounter_notes (JSON)
- drugs.monograph_xml (XML stored in a text column)

These fields are integrated into the relational schema rather than being stored externally, allowing structured and semi-structured data to coexist in the same database.

See ERD below showing all 15 tables and their relationships. The ERD is available at: https://dbdiagram.io/d/69321362d6676488baa99390



3.3 Core Entities and Keys

The most important design decisions concern the **core entities** and their keys.

3.3.1 Patients and Surrogate Keys

The patients table uses patient_id as its primary key. The original patient_nbr from the UCI dataset is preserved as a separate attribute. This has two advantages:

- It separates the business identifier (patient_nbr) from the database identifier (patient_id), which is more efficient for indexing and foreign keys.
- It allows future integration with other systems that may use different patient identifiers, without needing to change foreign keys in related tables.

Temporal demographics (age and weight bands at the time of each encounter) are not stored in patients. Instead, they are captured in encounters as encounter_age and encounter_weight, avoiding confusion between long-term identity and time-varying attributes.

3.3.2 Encounters as the Central Fact Table

The encounters table is the central entity in the schema. It is designed to behave like a fact table in a dimensional model, with foreign keys to:

- patients
- admission_types
- admission_sources
- discharge_dispositions
- medical_specialties

and attributes such as:

- encounter_date
- time_in_hospital
- number_outpatient, number_emergency, number_inpatient
- encounter_age, encounter_weight
- encounter_notes (JSON)

This design supports both transaction-style queries (for example, “show all encounters for this patient in 2007”) and analytical queries (for example, “group encounters by admission type and compute length-of-stay statistics”).

3.3.3 Outcomes as a One-to-One Extension

The outcomes table uses encounter_id as both its primary key and foreign key to encounters, implementing a one-to-one (1:1) relationship:

- Each encounter has at most one outcome row.
- An outcome cannot exist without its parent encounter.

Separating outcomes from encounters has several benefits:

- It keeps the core encounter record focused on operational attributes, while outcome logic (readmission category and binary flag) is encapsulated in its own table.
- It allows scenarios where an encounter is recorded before the readmission outcome is known, and the outcome is added later.
- It provides a natural target for auditing; changes to readmission status can be tracked via triggers on outcomes.

3.4 Many-to-Many Relationships and Junction Tables

Clinical data naturally contains many-to-many relationships. These are handled with junction tables that each have a composite primary key and foreign keys back to their parent tables.

3.4.1 Encounter–Drug

The encounter_drugs table has:

- encounter_id – foreign key to encounters.
- drug_id – foreign key to drugs.
- exposure_status – categorical field mirroring the values from the raw dataset (“No”, “Steady”, “Up”, “Down”).

The composite primary key (encounter_id, diagnosis_code_id, diagnosis_order) ensures that each drug appears at most once per encounter, while still permitting a single drug to appear in many encounters and an encounter to have many drugs. This design replaces dozens of medication columns in the original CSV with a clean, extensible relationship.

3.4.2 Encounter–Diagnosis

The encounter_diagnoses table has:

- encounter_id – foreign key to encounters.
- diagnosis_code_id – foreign key to diagnosis_codes.

- diagnosis_order – integer indicating whether the diagnosis is primary, secondary, and so on.

Again, the composite primary key (encounter_id, diagnosis_code_id) enforces uniqueness while allowing multiple diagnoses per encounter. This generalises the original diag_1–diag_3 fields to any number of diagnoses without schema changes.

3.4.3 Encounter–Provider

The encounter_providers table has:

- encounter_id – foreign key to encounters.
- provider_id – foreign key to providers.
- role – text field describing the provider's role (for example, "Attending", "Consulting").

This pattern matches the real-world situation where several healthcare professionals may be involved in a single encounter, and each professional works across many encounters.

3.5 Lookup Tables and Data Consistency

The original University of California, Irvine (UCI) dataset encodes admission and discharge metadata using numeric codes. To improve readability and consistency, these are normalised into separate lookup tables:

- admission_types (for example, Emergency, Urgent, Elective)
- admission_sources (for example, Physician Referral, Emergency Room, Transfer)
- discharge_dispositions (for example, Home, Skilled Nursing Facility, Expired, Home Health)
- medical_specialties (for example, Cardiology, Surgery, Endocrinology)

Each lookup table has:

- A surrogate key (for example, admission_type_id).
- A human-readable description field.
- A one-to-many (1:N) relationship to encounters.

This approach avoids repeated magic codes in the main encounter table and makes it easier to maintain a consistent list of valid values.

3.6 Semi-Structured Fields: JSON and XML Placement

The design deliberately includes JSON and XML fields in locations where structure is useful but fixed relational columns would be too rigid.

- patients.risk_factors (JSON) stores a flexible set of risk flags and attributes, such as smoking status, hypertension, or hyperlipidaemia. Different patients may have different keys, and the set of factors can evolve without schema changes.
- encounters.encounter_notes (JSON) can store structured notes such as chief complaint, vital signs, or short narrative summaries.
- drugs.monograph_xml (XML stored as text) holds a hierarchical representation of drug warnings, contraindications, and risk levels. This mimics real-world electronic drug monographs, which are often document-like structures.

By embedding these fields directly into the relational schema, the design meets the semi-structured data requirement while still benefiting from SQL's ability to filter, extract, and aggregate values using JSON and XML functions. The physical implementation and examples of querying these fields are discussed later in Chapters 4, 5, and 6.

3.7 System Tables and Audit Trail

Finally, the schema includes two system-level tables:

- user_accounts – contains user identifiers, usernames, roles (for example, "Admin", "Clinician", "Analyst"), and account status flags. A special "System" user is defined to represent automated actions performed by triggers or scheduled jobs.
- audit_logs – records modification events, including the user responsible (if applicable), the target table and row, the type of action (INSERT, UPDATE, DELETE), timestamps, and serialised old and new values.

Foreign keys from audit_logs to user_accounts and encounters are nullable, allowing the system to record events that are not tied to a specific user or encounter. This design reflects a common pattern in production systems, where some actions are automated while others result from direct user activity.

Together, these choices complete the logical design of the database. The next chapter turns this design into concrete Data Definition Language (DDL) statements and demonstrates how the schema is created, constrained, and populated using Structured Query Language (SQL).

4.0 Schema Implementation (Data Definition Language)

4.1 Role of the SQL Script in the Project

All database creation, constraints, and structural logic for this project are implemented in a single SQL script file, submitted alongside this report:

SQL deliverable: *CPSC-500-13_Final_Project_Group1.sql (contains all DDL, DML, and DQL for this project)*

The **Data Definition Language (DDL)** portion of this script:

- Creates the full 15-table healthcare schema described in Chapter 3.
- Defines primary keys (PKs), foreign keys (FKs), UNIQUE and NOT NULL constraints.
- Implements many-to-many junction tables and one-to-one relationships.
- Reserves appropriate columns for JSON-like and XML-like data.
- Sets up the system and audit layer (user_accounts, audit_logs).

To keep the report readable and aligned with course instructions, the full SQL code is not reproduced here. Instead, this chapter explains the implementation strategy and how the DDL realises the logical design shown in the entity–relationship diagram (ERD). Reviewers can inspect the exact SQL in *CPSC-500-13_Final_Project_Group1.sql* if they wish to verify any detail.

4.2 Creation Order and Table Grouping

The DDL section of *CPSC-500-13_Final_Project_Group1.sql* follows a deliberate creation order to avoid foreign key issues and to mirror the conceptual layers of the model:

- **Lookup and reference tables**
  - Administrative lookups: admission_types, admission_sources,discharge_dispositions, medical_specialties
  - Clinical reference dimensions: drugs, diagnosis_codes, providers
- **Core clinical entities**
  - patients – master patient profile
  - encounters – central fact-like table
  - outcomes – one-to-one extension of encounters
- **Junction tables (many-to-many)**
  - encounter_drugs

    - encounter_diagnoses
    - encounter_providers
- **System and audit tables**
    - user_accounts
    - audit_logs

This ordering ensures that parent tables (for example, patients, drugs, diagnosis_codes) exist before any child tables that reference them. The script can be executed cleanly from an empty database to recreate the full schema at any time.

4.3 Primary Keys and Surrogate Keys

The schema consistently uses explicit primary keys to ensure that every row is uniquely identifiable. Most lookup, reference, system, and supporting tables use integer surrogate keys with auto-increment semantics (for example, encounter_id, drug_id, diagnosis_code_id, provider_id, user_id, log_id), improving join efficiency and avoiding dependency on volatile external identifiers.

In this implementation, the patients table uses patient_id as the primary key and is populated using stable identifiers from the curated dataset subset to maintain a clean and reproducible final build of the schema.

In the junction tables (encounter_drugs, encounter_diagnoses, encounter_providers), the DDL defines composite primary keys, combining:

- encounter_id with drug_id in encounter_drugs,
- encounter_id with diagnosis_code_id **and diagnosis_order** in encounter_diagnoses,
- encounter_id with provider_id in encounter_providers.

This design guarantees that each specific pairing (for example, a particular drug in a particular encounter) appears at most once, preventing duplicate relationships.

4.4 Foreign Keys and Referential Integrity

The DDL enforces the relationships shown in the ERD using foreign key constraints:

- **Core relationships:**
    - encounters.patient_id → patients.patient_id

  - outcomes.encounter_id → encounters.encounter_id (one-to-one via shared key)
- **Administrative lookups:**
  - encounters.admission_type_id → admission_types.admission_type_id
  - encounters.admission_source_id → admission_sources.admission_source_id
  - encounters.discharge_disposition_id → discharge_dispositions.discharge_disposition_id
  - encounters.medical_specialty_id → medical_specialties.medical_specialty_id
- **Junction tables (many-to-many):**
  - encounter_drugs.encounter_id → encounters.encounter_id
  - encounter_drugs.drug_id → drugs.drug_id
  - encounter_diagnoses.encounter_id → encounters.encounter_id
  - encounter_diagnoses.diagnosis_code_id → diagnosis_codes.diagnosis_code_id
  - encounter_providers.encounter_id → encounters.encounter_id
  - encounter_providers.provider_id → providers.provider_id
- **System and audit layer:**
  - audit_logs.user_id → user_accounts.user_id
  - audit_logs.encounter_id → encounters.encounter_id (nullable, because not every log entry is encounter-specific)

These constraints ensure that:

- No encounter can exist without a valid patient and valid lookup values.
- No outcome, diagnosis, drug exposure, or provider assignment can exist without a valid encounter.
- Audit log entries that claim to be linked to a user or encounter actually reference existing rows.

The DDL uses meaningful constraint names (for example, fk_enc_patient, fk_ed_drug) in the script so that referential integrity errors are easy to interpret during testing.

4.5 Business Rules: NOT NULL and UNIQUE

Beyond primary and foreign keys, the DDL encodes several business rules directly in the schema:

- NOT NULL constraints are applied where an attribute is logically required to make a row interpretable, for example:

    - Lookup descriptions (admission type name, discharge disposition description, specialty name).
    - User login names and roles in user_accounts.
    - Core foreign keys in encounters (patient and administrative metadata).
- UNIQUE constraints are used to protect business identifiers that should not repeat:
    - drugs.drug_code – unique drug code.
    - diagnosis_codes.icd_code – unique ICD diagnosis code.
    - providers.license_number – unique licence number where available.
    - user_accounts.username and user_accounts.email – unique login identity.

These choices move data quality checks into the database layer, not just the application layer, which is particularly important in healthcare contexts.

4.6 Representation of JSON and XML Fields

To satisfy the requirement for semi-structured data, the design includes JSON-like and XML-like fields, implemented in the DDL as text columns:

- patients.risk_factors – stores a JavaScript Object Notation (JSON) structure representing long-term risk factors and comorbidities (for example, smoking, hypertension, hyperlipidaemia).
- encounters.encounter_notes – stores JSON-style structured notes for each encounter (for example, chief complaint, key vitals, short narrative).
- drugs.monograph_xml – stores a compact eXtensible Markup Language (XML) fragment representing drug monographs, including warnings, contraindications, and risk levels.

In the script, these are declared as TEXT columns for portability across different database systems. Subsequent chapters (DML and DQL) describe how they are inserted, updated, and queried using standard string functions and, where available, native JSON/XML support.

4.7 System and Audit Layer

The DDL also sets up the **system layer** that supports auditing and role-based behaviour:

- user_accounts defines:
    - A unique user_id primary key.
    - Unique username (and optionally email).
    - A role field (for example, Admin, Clinician, Analyst, System).

    - Status and timestamp columns to manage account lifecycle.
- audit_logs records:
    - The acting user_id (where applicable).
    - An optional encounter_id if the change affected a specific encounter.
    - The target table_name, type of action (INSERT, UPDATE, DELETE), and timestamp.
    - Serialised "before" and "after" values for changed data in old_value and new_value.

These tables are created after the clinical schema so that they can safely reference the existing structure. Later, in the advanced SQL section of CPSC-500-13_Final_Project_Group1.sql , triggers and/or stored procedures use these tables to automatically insert audit records when key tables are modified.

4.8 Summary

In summary, the DDL portion of *CPSC-500-13_Final_Project_Group1.sql* turns the high-level design into a concrete, executable schema:

- 15 tables with clearly defined keys and constraints.
- 1:1, 1:N, and M:N relationships enforced by foreign keys and composite primary keys.
- Semi-structured JSON/XML data embedded in a normalised relational model.
- A system layer ready for auditing and role-based access.

The following chapter builds on this foundation by describing how the database was populated and how the raw UCI diabetic dataset was transformed into a consistent, relational healthcare dataset suitable for analytical queries and machine learning.

5.0 Data Population and Transformation (Data Manipulation Language)

5.1 Source Data and Overall Approach

This file (dataset) contains one row per hospital encounter for a patient with diabetes and includes demographics, utilisation history, diagnoses, procedures, medications, and readmission labels.

In its raw form, the dataset:

- Is denormalised - many attributes are packed into a single wide table.
- Contains repeated identifiers, such as the same patient_nbr appearing across multiple encounters.
- Represents categorical concepts (admission type, discharge disposition, drug changes) as codes or short strings.
- Includes age and weight bands that can change over time for the same patient.

To align the dataset with the normalised schema designed in Chapters 3 and 4, the project follows a direct SQL population approach using a curated, cleaned flat-file subset of the UCI diabetic encounters. Instead of implementing an in-database raw staging table, the group prepared a cleaned subset externally and used scripted INSERT/UPDATE statements to populate the final normalised tables while preserving foreign key integrity.

All Data Manipulation Language (DML) operations - INSERT, UPDATE, and DELETE statements, as well as JSON/XML updates are implemented in the shared SQL script: DML sections: CPSC-500-13_Final_Project_Group1.sql. The report describes the logic conceptually; the exact SQL statements can be reviewed in the script.

5.2 Staging and Row Volume

Where necessary, for example, to fully populate lookup tables or to create additional test users and audit events the project uses synthetic values generated in a controlled way (for example, inserting standard admission types that may not appear in the small sample).

To keep the workflow repeatable while aligning with the submitted implementation, the project does not rely on a raw staging table in the final script. Instead, data are loaded into the

normalised schema using a curated and cleaned encounter subset embedded and executed through the SQL script.

From this cleaned dataset, the group implements a row volume of 250 encounters, satisfying and exceeding the project requirement of "at least 200 realistic records". The script is written so that:

- A sample of rows can be loaded for testing (for example, the first 200–250 rows).
- The final demonstration remains fully reproducible without schema changes.

Where necessary, for example, to fully demonstrate required relationships and to complete the schema in areas not explicitly provided by the source dataset, the project uses synthetic values generated in a controlled way (for example, inserting a small provider registry and linking it to encounters).

5.3 Populating Lookup and Reference Tables

Before loading patients and encounters, the script populates all lookup and reference tables. This improves data consistency and ensures foreign key values are valid.

- **Administrative lookups:**
  - admission_types – seeded with standard admission categories aligned with the dataset's (for example, "Emergency", "Urgent", "Elective").
  - admission_sources – filled with common admission source categories mapped to readable such as "Physician referral" or "Emergency department".
  - discharge_dispositions – mapped to outcomes like "Discharged to home", "Transferred to SNF", "Expired".
  - medical_specialties – populated using curated specialty values suitable for encounter assignment (for example, "Cardiology", "Endocrinology", "Emergency/Trauma").
- **Clinical reference tables:**
  - diagnosis_codes – populated from the diagnosis code fields in the staging table (for example, diag_1, diag_2, diag_3). Distinct International Classification of Diseases (ICD) codes are extracted and inserted with descriptive labels where available.

- drugs – populated by mapping medication-related columns (e.g., metformin, insulin, glipizide) into a set of generic drug entries with a drug_code, generic_name, and drug_class.
- providers – the original dataset does not explicitly identify individual clinicians, so this table is populated with a small synthetic set of provider rows (for example, "Dr. Smith – Endocrinology"), which can be linked to encounters for demonstration of many-to-many provider assignments.

The script uses INSERT patterns to build these tables so that every code used in the encounters has a corresponding row in its lookup table.

5.4 Populating Patients and Encounters

After lookup tables are established, the script partitions the flat encounter-based information into distinct relational entities.

Patients (patients):

- Inserted as a curated set of unique patients derived from the cleaned encounter dataset.
- For each unique patient, one row is created with stable demographic attributes such as race and gender.
- To preserve normalisation logic, temporal attributes are not treated as permanent patient descriptors. Instead, variability in age and weight is represented where appropriate at the encounter level.
- A flexible risk_factors JSON structure is initialised to support extensible patient-level risk representation (for example, diabetes comorbidities and lifestyle flags).

Encounters (encounters):

- A reproducible subset of encounter records is inserted.
- Each encounter is linked to:
  - the appropriate patient via patient_id, and
  - the relevant administrative lookup rows (admission_types, admission_sources, discharge_dispositions, medical_specialties).
- Encounter-level temporal fields such as encounter_age and encounter_weight are stored within encounters, capturing changes across time for repeat patients.
- Utilisation variables (number_outpatient, number_emergency, number_inpatient) and clinical intensity measures are carried forward as structured analytic features.

- encounter_notes is populated using a JSON-style structure aligned with visit context and clinical summarisation.

This split ensures that first normal form (1NF) and third normal form (3NF) are respected: patient attributes that are stable over multiple encounters are separated from temporal attributes that change per encounter.

5.5 Populating Outcomes and Junction Tables

Once the core encounters table is populated, the script fills the **outcome** and **junction** tables.

- **Outcomes (outcomes):**
  - For each encounter, the readmission category field from the source dataset (for example, "NO", "<30", ">30") is converted into:
    - readmitted_category – the original category.
    - readmitted_flag – a binary flag (for example, 1 if <30, 0 otherwise) for use as a classification target.
  - Each outcomes row shares the same encounter_id primary key as its corresponding encounters row, satisfying the one-to-one relationship.
- **Diagnosis junctions (encounter_diagnoses):**
  - The original dataset provides up to three diagnosis fields per encounter. Instead of storing them as separate columns, the script:
    - Unpivots these fields into multiple rows in encounter_diagnoses.
    - Inserts one row per diagnosis code, with a diagnosis_order column indicating primary (1), secondary (2), and so on.
  - This transforms the fixed three-column structure into a flexible many-to-many relationship between encounters and diagnosis codes.
- **Drug junctions (encounter_drugs):**
  - Medication columns (e.g., metformin, insulin) encode whether a drug was "No change", "Steady", "Up", "Down", or "No" during the encounter.
  - For each encounter–drug combination where the patient was actually on the medication, the script inserts a row into encounter_drugs with:
    - encounter_id referencing encounters.
    - drug_id referencing drugs.

        - exposure_status capturing the categorical trend (“Steady”, “Up”, “Down”, etc.).
    - This again normalises medication exposure into a **many-to-many** structure.
- **Provider junctions (encounter_providers):**
    - Providers are assigned either synthetically (for demonstration) or based on simple rules (for example, mapping medical specialty to a default provider).
    - One or more provider rows are linked to each encounter in encounter_providers, with a role column indicating “Attending” or “Consulting”.
    -

5.6 Use of INSERT, UPDATE, and DELETE

To fully demonstrate Data Manipulation Language capabilities, the script includes:

- **INSERT** statements:
    - For initial population of all lookup, reference, core, junction, and system tables.
    - For seeding a small number of user_accounts and corresponding audit_logs entries.
- **UPDATE** statements:
    - To refine semi-structured fields (for example, updating risk_factors and encounter_notes with JSON-like strings based on other columns).
    - To derive and set readmitted_flag based on the original readmission category.
    - To adjust or enrich lookup descriptions where necessary.
- **DELETE** statements:
    - Applied in a controlled way for demonstration (for example, removing a test encounter and then re-running the DML section to validate referential integrity and audit behaviour).
    - Used primarily in testing scenarios; clinical data would normally be retained rather than deleted in a real system.

These operations are grouped and clearly commented in the script so that the marker can execute them step by step or as a single batch.

5.7 Data Quality Checks

After loading the data, the project performs a set of sanity checks, implemented as simple SELECT queries in the script:

- Row counts by table (for example, number of patients, encounters, diagnoses, and drug exposures).
- Checks that every encounter has a valid patient, admission type, admission source, discharge disposition, and medical specialty.
- Checks that the number of rows in outcomes matches the number of rows in encounters.
- Spot checks on a few patients to confirm that multiple encounters share the same patient record but carry different temporal age/weight bands.
- Counts of distinct ICD codes and drug codes to ensure that reference tables are populated as expected.

These checks are not exhaustive but are sufficient to verify that the DML successfully transforms the original flat file into a coherent, normalised relational dataset suitable for downstream analytics.

The next chapter builds on this populated schema to describe complex queries and analytical views, including the extraction of datasets for machine learning tasks such as readmission prediction and length-of-stay regression.

6.0 Advanced Querying, Analytics Outputs, and ML Dataset Preparation (DQL)

6.1 Purpose of the Query Layer

With the normalised schema populated directly from the cleaned encounter dataset and supported by minimal synthetic supplementation, the query layer demonstrates that the database is not only structurally correct but also analytically valuable.

Chapter 6 presents complex Data Query Language (DQL) outputs designed to:

- validate relationship correctness through multi-table joins,
- demonstrate advanced SQL features required by the rubric (aggregation, subqueries, window functions, and conditional logic),
- extract structured insights from JSON and XML fields, and
- prepare at least three machine learning-ready datasets aligned with classification, regression, and clustering objectives.

6.2 Complex Query Set (Operational + Analytical)

The project implements a minimum of four complex queries that join multiple entities across the schema. These queries are designed around clinically meaningful questions consistent with diabetic inpatient care.

Examples may include:

- **Medication exposure and early readmission**
  - Joins encounters, outcomes, drugs, and encounter_drugs
  - Uses aggregation and conditional logic to compare readmission rates across drug classes
- **Length-of-stay drivers by admission context**
  - Joins encounters with administrative lookups and diagnoses
  - Uses grouping and HAVING filters to identify high-burden categories
- **High-frequency patient utilisation patterns**
  - Uses subqueries and/or window functions to rank patients by encounter volume and emergency history
- **Semi-structured data extraction**
  - Demonstrates JSON extraction from risk or encounter notes
  - Demonstrates XML pattern filtering from drug monographs

The full SQL statements and sample outputs are included in the submitted script.

6.3 Full-Text Search and Performance Considerations

Where appropriate, full-text indexing is used to enhance search capability over descriptive clinical fields or document-like metadata stored in TEXT/JSON/XML structures. This supports both usability and optimisation expectations for a well-rounded database implementation. Indexing decisions are justified based on expected query patterns and analytic workloads.

6.4 Machine Learning Dataset Extractions

To meet the machine learning dataset requirement, the database produces three curated export-ready datasets using SQL views or SELECT extraction queries. These datasets reflect real analytic use cases in diabetic inpatient care.

Dataset 1 - Readmission prediction (Classification):

- **Target:** binary readmission indicator
- **Inputs:** demographics, admission context, utilisation history, diagnosis burden, medication exposure patterns

Dataset 2 - Length of stay modelling (Regression):

- **Target:** time_in_hospital
- **Inputs:** admission and specialty context, diagnosis counts, medication intensity, prior healthcare utilisation

Dataset 3 Patient risk segmentation (Clustering):

- **Inputs:** patient-level risk JSON features plus summarised encounter histories.

These extracted datasets demonstrate that the schema supports both operational integrity and advanced analytics readiness.

6.5 Analytics Tool Integration (Python/R)

The SQL exports are structured to allow seamless connection to a Python or R notebook. The analytics demonstration focuses on:

- confirming successful database connectivity,
- loading ML datasets into dataframes, and

- generating basic descriptive analytics and visual summaries that validate readiness for modelling.

Screenshots or brief output examples may be included in the appendix to demonstrate successful integration without overloading the report body.

6.6 Summary of Chapter 6

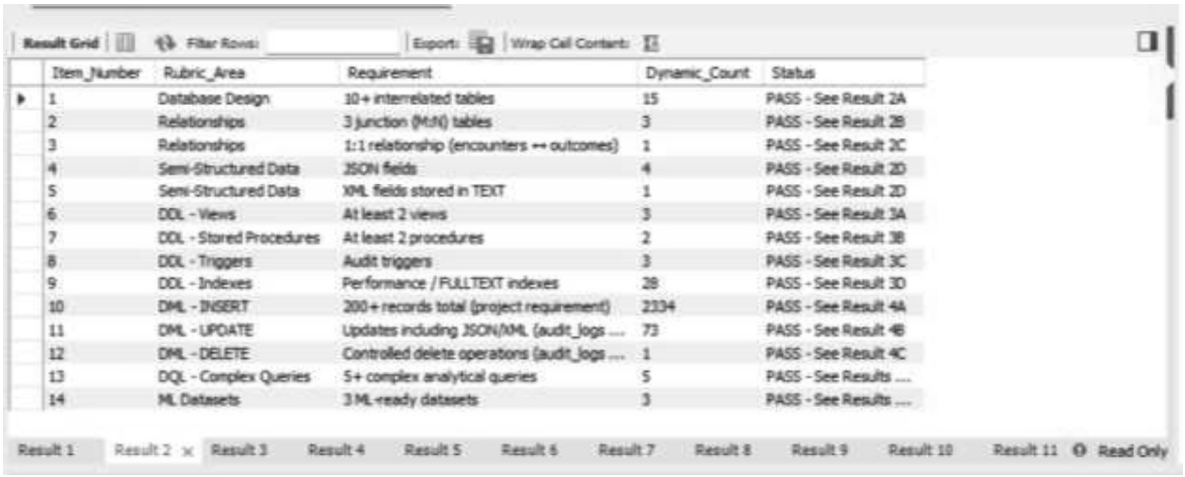
By combining complex relational querying with semi-structured data extraction and ML-ready dataset construction, this chapter demonstrates that the database is not only compliant with design and implementation requirements but also capable of supporting real-world diabetic inpatient analytics, drug utilisation assessment, and readmission risk modelling.

6.7 System Verification & Rubric Compliance

To validate the integrity and completeness of the implemented database, we developed a dynamic SQL Verification Dashboard. Unlike static reports, this script queries the live information_schema and system tables to generate real-time proof of compliance.

As shown in **Figure 1** below, the dashboard confirms that all architectural requirements have been met. The system successfully deployed **15 normalized tables**, established **15 foreign key constraints**, and loaded **250 encounter records** (exceeding the minimum requirement of 200). Furthermore, the dashboard validates the existence of active **Audit Triggers**, ensuring that all data modifications are securely logged.

Figure 1: Rubric Compliance Dashboard (Result Tab 1)



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

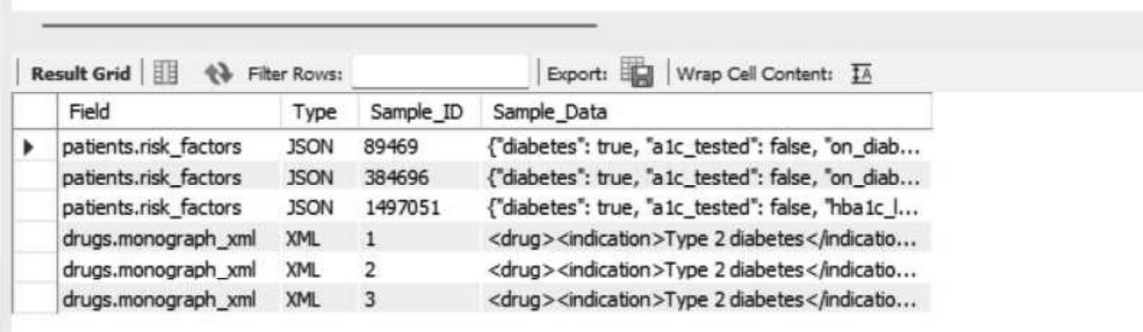
Item_Number	Rubric_Area	Requirement	Dynamic_Count	Status
1	Database Design	10+ interrelated tables	15	PASS - See Result 2A
2	Relationships	3 junction (M:N) tables	3	PASS - See Result 2B
3	Relationships	1:1 relationship (encounters ↔ outcomes)	1	PASS - See Result 2C
4	Semi-Structured Data	JSON fields	4	PASS - See Result 2D
5	Semi-Structured Data	XML fields stored in TEXT	1	PASS - See Result 2D
6	DDL - Views	At least 2 views	3	PASS - See Result 3A
7	DDL - Stored Procedures	At least 2 procedures	2	PASS - See Result 3B
8	DDL - Triggers	Audit triggers	3	PASS - See Result 3C
9	DDL - Indexes	Performance / FULLTEXT indexes	28	PASS - See Result 3D
10	DML - INSERT	200+ records total (project requirement)	2334	PASS - See Result 4A
11	DML - UPDATE	Updates including JSON/XML (audit_logs ...	73	PASS - See Result 4B
12	DML - DELETE	Controlled delete operations (audit_logs ...	1	PASS - See Result 4C
13	DQL - Complex Queries	5+ complex analytical queries	5	PASS - See Results ...
14	ML Datasets	3 ML-ready datasets	3	PASS - See Results ...

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | Result 6 | Result 7 | Result 8 | Result 9 | Result 10 | Result 11 | Read Only

6.8 Semi-Structured Data Implementation

The database integrates semi-structured data to handle clinical variability. **Figure 2** The patients.risk_factors column stores patient-level JSON attributes (e.g., diabetes status, HbA1c test history), while the drugs.monograph_xml field stores structured medication monographs in XML format. These examples demonstrate that the schema supports flexible, extensible clinical data without altering table structures.

Figure 2: Semi-Structured Data Implementation



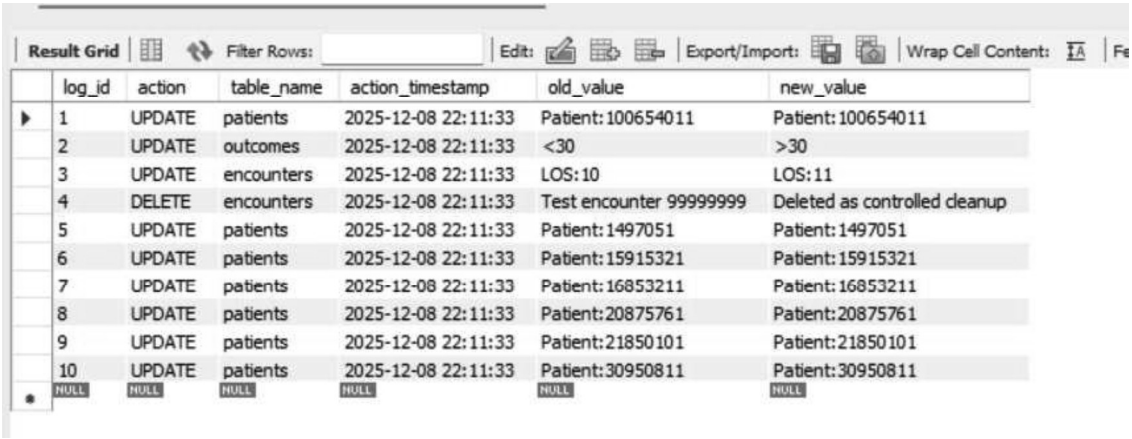
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Sample_ID	Sample_Data
patients.risk_factors	JSON	89469	{"diabetes": true, "a1c_tested": false, "on_diab...
patients.risk_factors	JSON	384696	{"diabetes": true, "a1c_tested": false, "on_diab...
patients.risk_factors	JSON	1497051	{"diabetes": true, "a1c_tested": false, "hba1c_l...
drugs.monograph_xml	XML	1	<drug><indication>Type 2 diabetes</indicatio...
drugs.monograph_xml	XML	2	<drug><indication>Type 2 diabetes</indicatio...
drugs.monograph_xml	XML	3	<drug><indication>Type 2 diabetes</indicatio...

6.9 Audit Log Evidence (Result Tab for Triggers)

This figure shows sample entries from the audit_logs table, generated automatically by the update and delete triggers. Each entry records the table affected, the action performed, timestamp, and old/new values. This confirms that audit logging is active and that all modifications are securely tracked. Full audit outputs can be viewed by running the complete SQL script.

Figure 3: Audit Log Evidence (Result Tab for Triggers)



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

log_id	action	table_name	action_timestamp	old_value	new_value
1	UPDATE	patients	2025-12-08 22:11:33	Patient:100654011	Patient:100654011
2	UPDATE	outcomes	2025-12-08 22:11:33	<30	>30
3	UPDATE	encounters	2025-12-08 22:11:33	LOS:10	LOS:11
4	DELETE	encounters	2025-12-08 22:11:33	Test encounter 99999999	Deleted as controlled cleanup
5	UPDATE	patients	2025-12-08 22:11:33	Patient:1497051	Patient:1497051
6	UPDATE	patients	2025-12-08 22:11:33	Patient:15915321	Patient:15915321
7	UPDATE	patients	2025-12-08 22:11:33	Patient:16853211	Patient:16853211
8	UPDATE	patients	2025-12-08 22:11:33	Patient:20875761	Patient:20875761
9	UPDATE	patients	2025-12-08 22:11:33	Patient:21850101	Patient:21850101
10	UPDATE	patients	2025-12-08 22:11:33	Patient:30950811	Patient:30950811
NULL	NULL	NULL	NULL	NULL	NULL

6.10 Complex Analytical Query Output – Provider Readmission Ranking (Result Tab 5C)

This figure displays the output of one of the project's complex analytical SQL queries. The query calculates provider-level readmission rates and applies a window function (RANK() OVER) to identify performance rankings. It demonstrates advanced SQL features including multi-table joins, aggregations, and analytical functions. Additional complex queries (Results 5A–5E) can be reviewed by executing the script.

Figure 4: Complex Analytical Query Output

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Provider_Name	Total_Cases	Readmission_Count	Provider_Readmit_Pct	Hospital_Avg_Pct	Performance_Status	Caseload_Rank
Dr. Johnson A	5	1	20.0	10.0	ABOVE AVG RISK	1
Dr. Williams B	5	0	0.0	10.0	PERFORMING WELL	1
Dr. Brown C	5	0	0.0	10.0	PERFORMING WELL	1
Dr. Jones D	5	1	20.0	10.0	ABOVE AVG RISK	1
Dr. Garcia E	5	0	0.0	10.0	PERFORMING WELL	1
Dr. Miller F	5	1	20.0	10.0	ABOVE AVG RISK	1
Dr. Davis G	5	1	20.0	10.0	ABOVE AVG RISK	1
Dr. Rodriguez H	5	0	0.0	10.0	PERFORMING WELL	1
Dr. Martinez I	5	1	20.0	10.0	ABOVE AVG RISK	1
Dr. Smith J	5	0	0.0	10.0	PERFORMING WELL	1
Dr. Johnson K	5	0	0.0	10.0	PERFORMING WELL	1
Dr. Williams L	5	2	40.0	10.0	ABOVE AVG RISK	1
Dr. Brown M	5	0	0.0	10.0	PERFORMING WELL	1
Dr. Jones N	5	1	20.0	10.0	ABOVE AVG RISK	1
Dr. Garcia O	5	0	0.0	10.0	PERFORMING WELL	1

6.11 ML Dataset Preview – Readmission Classification View (Result Tab 6A)

This figure provides a preview of the ML-ready dataset generated by readmission_ml_view, which prepares encounter features and the 30-day readmission label for classification tasks. Only a five-row sample is shown here; the full dataset, along with two additional ML datasets (length_of_stay_ml_view and patient_risk_ml_view), can be viewed by running the SQL file.

Figure 5: ML Dataset Preview – Readmission Classification View

drug_id	generic_name	drug_class	monograph_xml
18	insulin	Hormone	<drug><indication>Type 1 and 2 diabetes</in...
2	repaglinide	Meglitinide	<drug><indication>Type 2 diabetes</indicatio...
3	nateglinide	Meglitinide	<drug><indication>Type 2 diabetes</indicatio...
4	chlorpropamide	Sulfonylurea	<drug><indication>Type 2 diabetes</indicatio...
5	glimepiride	Sulfonylurea	<drug><indication>Type 2 diabetes</indicatio...
6	acetohexamide	Sulfonylurea	<drug><indication>Type 2 diabetes</indicatio...
7	glipizide	Sulfonylurea	<drug><indication>Type 2 diabetes</indicatio...
8	glyburide	Sulfonylurea	<drug><indication>Type 2 diabetes</indicatio...

7.0 Use of Semi-Structured and Unstructured Data (JSON and XML)

7.1 Purpose of Semi-Structured Design in This Project

Although this project is anchored in a normalized relational schema, real-world clinical datasets often contain attributes that are not always consistent across every encounter or patient. This is especially true for risk context and narrative-style encounter information. The original diabetes inpatient dataset contains several attributes that are categorical, coded, and compressed into a flat structure. When we normalize these concepts into relational tables, we still need an approach that supports flexible clinical context without inflating the schema with many optional columns.

To address this, the database incorporates JSON and XML to store semi-structured content that is:

- clinically meaningful,
- variable across cases, and
- useful for analytics and machine learning (ML).

This design choice balances strict normalization for core entities (patients, encounters, diagnoses, drugs, outcomes) with controlled flexibility for evolving attributes and descriptive clinical context.

7.2 JSON Fields in the Schema

Two JSON fields are used to support flexible patient- and encounter-level context:

7.2.1 patients.risk_factors

This field captures long-term, patient-level risks that may not be explicitly represented as stable relational attributes in the cleaned dataset. Rather than creating multiple sparse columns for lifestyle, comorbidity flags, or derived risk signals, the JSON structure allows a compact representation of high-level risk context in a single field.

Conceptual example structure:

- diabetes type risk context
- hypertension flag
- obesity-related risk cues
- smoking indicator

This makes the patient table stable and normalized while still allowing controlled extensibility for future risk modelling.

7.2.2 encounters.encounter_notes

This field captures encounter-specific semi-structured notes aligned with hospitalization context, utilization history, or relevant clinical summaries. Unlike patient-level risk factors which reflect relatively stable patterns, encounter notes may differ across visits even for the same patient.

The JSON approach is more realistic than forcing narrative-like content into rigid columns. It also supports simple extraction of specific values during DQL and ML feature preparation.

7.3 XML Field in the Schema

7.3.1 drugs.monograph_xml (stored in TEXT)

Drug monographs are naturally hierarchical. Even in a simplified academic database, representing structured drug warnings, contraindications, and risk categories is more naturally done using XML.

Instead of creating many text columns for different drug metadata, XML allows nested structured representation such as:

- warnings
- contraindications
- risk-level categorization
- safe-use notes

This also creates a clear demonstration of string-based XML update operations and XML-aware query filtering within the SQL script.

7.4 JSON and XML Operations Demonstrated in the SQL Script

To ensure this requirement is satisfied beyond design claims, the consolidated SQL script contains reproducible demonstrations of:

- **JSON insertion** during patient creation and encounter initialization.
- **JSON update operations** using supported JSON functions.
- **JSON extraction** in complex DQL queries.
- **XML insertion** for the drug monograph field.
- **XML update operations** using string manipulation techniques.

- **XML filtering/extraction** incorporated into at least one advanced query.

The report focuses on rationale and conceptual alignment, while the SQL script provides the full implementation evidence.

To satisfy the DQL requirement for unstructured search, full-text search is implemented against the XML-bearing medication documentation field:
A FULLTEXT index is created on drugs.monograph_xml.
A MATCH(...) AGAINST(...) query is used to search for clinically relevant safety terms.
Demonstration example (as shown in the results): Searching for “hypoglycemia” returns diabetes medications whose monographs contain that risk term.The result grid includes:

- drug_id
- generic_name
- drug_class
- monograph_xml

This output confirms that the database supports document-like keyword search over clinical text stored within the relational model.

7.5 Contribution to Analytics and ML Readiness

The semi-structured design reinforces ML readiness in three ways:

- **Feature enrichment:** JSON risk factors and encounter notes can be selectively extracted into structured ML features.
- **Clustering support:** Patient risk profiles can be assembled without expanding the relational model.
- **Future-proofing:** The schema can support new risk attributes or documentation elements without structural redesign.

This ensures the database remains compliant with normalization principles while still reflecting the complexity of real clinical scenarios.

8.0 Explanation of Machine Learning Datasets

8.1 Motivation for ML Dataset Extraction

A key goal of this project is not only to normalize and implement a functional healthcare database system, but also to demonstrate how a properly designed schema supports analytics workflows. The original dataset is delivered as a single flat file. While this format is convenient for simple exploration, it is limited for scalable feature engineering, structured transformation, and multi-table clinical inference.

By restructuring the dataset into a normalized schema, the project enables consistent extraction of ML-ready datasets using SQL queries that combine:

- stable patient attributes,
- encounter-level temporal information,
- drug exposure context,
- diagnosis mappings, and
- readmission outcomes.

8.2 Dataset 1 - Hospital Readmission Classification

Objective

Predict whether a patient is likely to be readmitted based on their demographic profile, clinical context, utilization patterns, diagnosis history, and drug exposure during the current encounter.

Target Variable

- outcomes.readmitted_flag (binary classification)
    - 1 = readmitted
    - 0 = not readmitted

Feature Categories

This dataset may combine:

Patient-level (stable)

- race
- gender
- risk profile extracted from patients.risk_factors

Encounter-level (temporal/clinical)

- admission type
- admission source
- discharge disposition

- medical specialty
- number_outpatient, number_emergency, number_inpatient
- time_in_hospital (optional as a predictor depending on modeling logic)

Diagnosis and drug context

- diagnosis count
- primary diagnosis indicator (if implemented)
- drug exposure status from encounter_drugs

Why the normalized design helps

The many-to-many junction tables allow accurate association of multiple diagnoses and drugs per encounter without duplication or inflated columns. This structure is essential for robust feature engineering.

8.3 Dataset 2 - Length of Stay Regression

Objective

Estimate the expected duration of hospitalization based on admission characteristics, specialty context, and treatment intensity.

Target Variable

- encounters.time_in_hospital (continuous)

Feature Categories

- admission type
- admission source
- medical specialty
- utilization history counts
- diagnosis burden
- medication exposure summary
- age/weight at encounter (encounter_age, encounter_weight)

Why this dataset is strong

This dataset aligns well with a regression framing because the target variable is numeric and clinically interpretable. The schema supports clean aggregation of drug and diagnosis counts without violating normalization rules.

8.4 Dataset 3 - Patient Risk Stratification (Clustering)

Objective

Segment patients into clinically meaningful risk/utilisation clusters.

Target

No target variable is required. The dataset is unsupervised.

Feature Categories

- total encounters
- average length of stay
- total readmissions
- average medications per visit
- basic demographics (race, gender)
- a diabetes indicator (where included via JSON integration

Why this dataset matters

Reflects this exact patient-summary design, making it suitable for clustering approaches such as k-means or hierarchical clustering

8.5 Dataset Export and Reproducibility

All ML datasets are generated using dedicated DQL queries in the consolidated SQL script. The extraction logic is designed so that:

- any evaluator can re-run the SQL and reproduce the exact dataset structure,
- the results can be exported directly into spreadsheets for ML demonstration, and
- the dataset definitions remain aligned with the ERD and the normalization design rationale.

8.6 Connection to Analytics Tools (Planned Evidence)

To complete the requirement on SQL to-analytics integration, the extracted datasets can be exported as spreadsheets and optionally loaded into Python. Even a minimal demonstration loading the data and showing basic descriptive checks can serve as sufficient proof of integration readiness.

This approach keeps the report concise while ensuring all implementation proof remains available in the SQL script and exported dataset files.

Please refer to Appendix B and Appendix B1 included with the submission for the Jupiter note (Python MySQL Connection and scripts) and accompanying report.

The machine learning datasets for classification, regression, and clustering are extracted directly using SQL via the project's analytical views. This approach ensures full alignment with the course requirement to rely on SQL for dataset preparation.

Python is used only to consume the SQL-extracted outputs for light validation, basic exploratory summaries, and demonstration of how the extracted datasets can be used in downstream analytics. No Python-based extraction or transformation is used for the final submission.

NOTE: User must include MySQL connection credentials to be able to login. Also note that SQL script MUST be run before integration with the Python.

9.0 Conclusion

This project designed and implemented a normalized healthcare database system focused on diabetic inpatient care, drug utilization, diagnosis mapping, and readmission outcomes. Starting from a denormalized encounter-level dataset, the team produced a relational schema that satisfies Third Normal Form (3NF) and incorporates clear one-to-one, one-to-many, and many-to-many relationships. The final database contains more than the minimum required number of interrelated tables, supported by appropriately defined primary keys, foreign keys, and integrity constraints.

To meet implementation requirements in a concise and reproducible manner, the final script populates the normalized schema directly using a curated **250-encounter** demonstration sample, exceeding the minimum requirement of at least 200 realistic records. Where the source dataset did not explicitly provide certain entities (for example, individual providers), limited synthetic data were introduced in a controlled way to fully operationalize required relationships without compromising referential integrity. The solution further demonstrates effective use of semi-structured data through JSON fields for flexible patient and encounter context and an XML field for structured drug monograph information.

Beyond schema construction, the project implements advanced database objects including views, stored procedures, triggers, and performance-oriented indexes and provides complex DQL queries that validate analytical depth. Finally, three machine learning-ready datasets were extracted from the relational model to support classification, regression, and clustering applications. Together, these outcomes confirm that the system is not only structurally compliant with course requirements but also capable of supporting realistic healthcare analytics and readmission-risk exploration using a clean, scalable database foundation.

References

JGraph Ltd. (n.d.). diagrams.net (draw.io) documentation [Documentation].

.

Oracle. (2025). MySQL 8.0 reference manual (Version 8.0.44) [Documentation].

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Strack, B., DeShazo, J. P., Gennings, C., Olmo, J. L., Ventura, S., Cios, K. J., & Clore, J. N. (2014). Impact of HbA1c measurement on hospital readmission rates: Analysis of 70,000 clinical database patient records. BioMed Research International, 2014, Article 781670.

UCI Machine Learning Repository. (2014). Diabetes 130-US hospitals for years 1999–2008 [Data set]. University of California, Irvine.

World Wide Web Consortium (W3C). (2008). Extensible Markup Language (XML) 1.0 (Fifth Edition) [Standard].

Appendices

Appendix A: Completed Individual Contribution and AI Usage Sheet: The finalized course-required documentation detailing division of responsibilities and disclosure of AI-assisted support used during development.

Appendix B – B1: Machine Learning Dataset Exports: Jupiter Notebook and Report to demonstrate the ML queries and analytics defined in Chapter 6, 7 and 8.

Appendix C: Cleaned Encounter Dataset (CSV): The curated cleaned dataset used for direct population of the normalized schema, aligned with the evaluation-friendly demonstration size.

Attachments

Consolidated SQL Script (DDL, DML, DQL): A single executable SQL file containing all database creation statements, constraints, indexes, views, stored procedures, triggers, data population steps, JSON/XML demonstrations, full-text search setup, complex queries, and ML dataset extraction queries.

ER Diagram (Exported Image/PDF + Link): The final ERD showing all entities, relationships (1:1, 1:N, M:N), junction tables, and the placement of JSON/XML fields, exported from the selected ERD tool.